

Functions

Where We Left Off

Variables, structs, and arrays hold data

Operators compute new values from them

Every statement so far has lived in **main**

Real programs have thousands of
statements

The Same Code, Three Times

```
psi1 = ( 2736.0f - counts1 ) / 25.6f ;
```

```
psi2 = ( 2736.0f - counts2 ) / 25.6f ;
```

```
psi3 = ( 2736.0f - counts3 ) / 25.6f ;
```

Find the calibration error - fix it three times
Miss one, and two readings are wrong

Group Statements Under a Name

A **function** is a named block of statements

Write it **once**

Call it by name as many times as you need

It can take inputs and give back a result

Why Functions?

- Reuse** write once, call many times
- Name an idea** `adc_to_psi` says what it does
- Test** check one small piece at a time
- Hide detail** the caller need not know how

Anatomy of a Function

float	return type
adc_to_psi	name
(uint32_t counts)	parameter list
{ ... }	body - the statements

```
float adc_to_psi ( uint32_t counts ) {  
    return ( 2736.0f - (float) counts ) / 25.6f ;  
}
```

A Function in C

```
#define ZERO_PSI_COUNTS 2736.0f
#define COUNTS_PER_PSI  25.6f

float adc_to_psi ( uint32_t counts ) {
    return ( ZERO_PSI_COUNTS - (float) counts )
        / COUNTS_PER_PSI ;
}
```

A Function in C++

```
constexpr float ZERO_PSI_COUNTS      = 2736.0f
;
constexpr float COUNTS_PER_PSI = 25.6f ;

float adc_to_psi ( uint32_t counts ) {
    return ( ZERO_PSI_COUNTS - (float) counts )
        / COUNTS_PER_PSI ;
}
```

The function itself is the same as C

A Function in Rust

```
const ZERO_PSI_COUNTS : f32 = 2736.0 ;  
const COUNTS_PER_PSI : f32 = 25.6 ;
```

```
fn adc_to_psi ( counts : u32 ) -> f32 {  
    ( ZERO_PSI_COUNTS - counts as f32 ) /  
    COUNTS_PER_PSI  
}
```

fn keyword, name : type, return type after ->

Returning a Value

C, C++, Rust:
return value ;

Rust also allows a final **expression**
with no semicolon:

```
fn square ( x : i32 ) -> i32 {  
    x * x  
}
```

Functions That Return Nothing

C, C++:

```
void show ( float psi ) {  
    printf ( "%.1f psi\n", psi ) ;  
}
```

Rust: leave off the ->

```
fn show ( psi : f32 ) {  
    println! ( "{:.1} psi", psi ) ;  
}
```

Calling a Function

```
psi = adc_to_psi ( 1456 ) ;      50.0
```

```
my_sample.psi = adc_to_psi  
( my_sample.adc_counts ) ;
```

```
show ( adc_to_psi ( 1200 ) ) ;   60.0 psi
```

Same syntax in C, C++, and Rust

Declare Before Use: C, C++

The compiler reads the file **top to bottom**

A call to a function it has not seen yet is an **error**

Fix it with a **prototype** - the signature alone:

```
float adc_to_psi ( uint32_t counts ) ;
```

Rust does not care about order

Prototypes Go in a Header

sensor.h

```
#ifndef SENSOR_H
#define SENSOR_H
float adc_to_psi ( uint32_t counts ) ;
#endif
```

main.c

```
#include "sensor.h"
```

Using extern in Prototypes

sensor.h

```
#ifndef SENSOR_H
#define SENSOR_H
extern float adc_to_psi ( uint32_t counts ) ;
#endif
```

main.c

```
#include "sensor.h"
```

Extern shows that definition is elsewhere
Not necessary but useful for easier reading

main is a Function Too

C: int main (void) { ... return 0 ; }

C++: int main () { ... return 0 ; }

Rust: fn main () { ... }

The operating system **calls** main

Its return value is the **exit status**
0 means success

Parameters and Arguments

Parameter: the name in the definition

```
float adc_to_psi ( uint32_t counts )
```

Argument: the value in the call

```
adc_to_psi ( 1456 )
```

When called: counts <= (1456)

Parameters are Local Variables

Each call gets **fresh** parameters

They exist only while the function runs

Variables declared inside are **local** too

When the function returns, they are gone

The Call Stack

Each call gets a **stack frame** in memory

frame: adc_to_psi

counts = 1456

psi = 50.0 (local)

return address

*pushed by the call
popped by return*

frame: main

my_sample

count

...

Parameters and locals vanish when the frame is popped

Scope

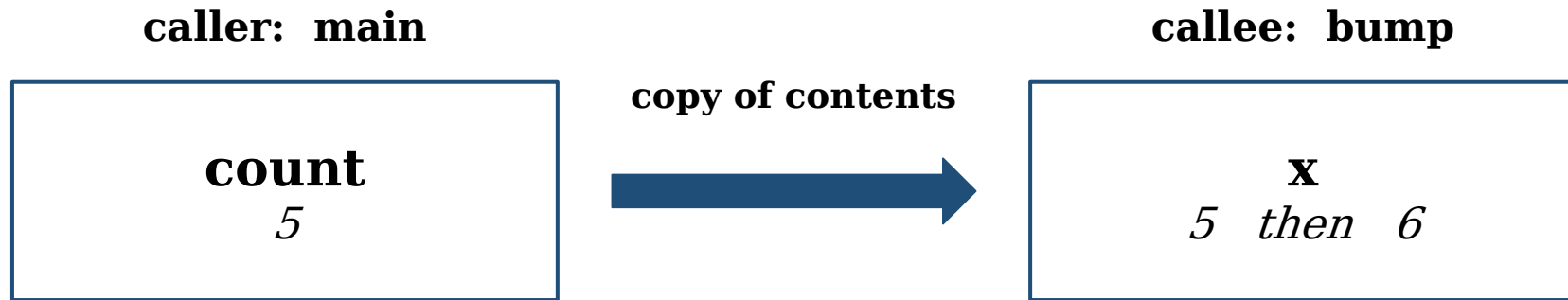
```
float adc_to_psi ( uint32_t counts ) {  
    float psi = ... ;           local  
    return psi ;  
}
```

main cannot see **counts** or **psi**

Two functions may each have a variable named psi
They are **different** variables

Pass by Value

The **contents** of the argument are copied into the parameter



`x <= ( count )` `count` stays 5

Pass by Value in C

```
void bump ( int x ) {  
    x = x + 1 ;  
}
```

```
int count = 5 ;  
bump ( count ) ;
```

count is **still 5**
bump changed its own copy

Pass a Pointer in C

```
void bump ( int *x ) {  
    *x = *x + 1 ;  
}
```

$((x)) \leq ((x)) + 1$

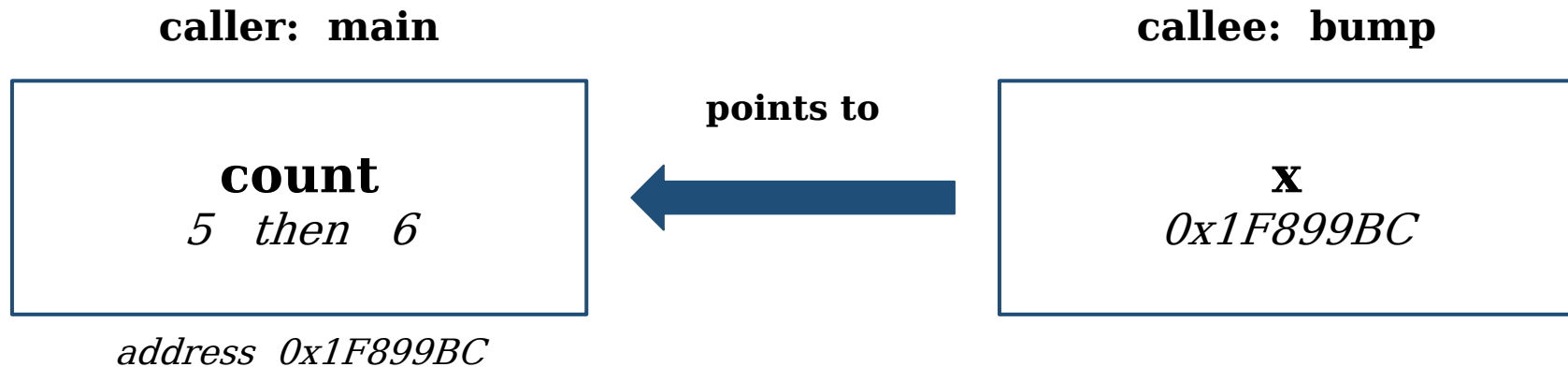
```
int count = 5 ;  
bump ( &count ) ;
```

$x \leq \&(\text{count})$

count is now **6**

What the Pointer Does

The parameter holds the **address** of count



`x <= &( count )` `((x)) <= ((x)) + 1`

Pass by Reference in C++

```
void bump ( int &x ) {  
    x = x + 1 ;  
}
```

```
int count = 5 ;  
bump ( count ) ;
```

count is now **6**
x is another **name** for count

The Hidden Danger

In C++ these two calls look **identical**:

<code>show (count) ;</code>	cannot change count
<code>bump (count) ;</code>	changes count

You must read the **signature** to know

In C, the `&count` at the call is a warning sign

Read-Only Access with const

Large structs: pass the address, but promise not to change it

C: void show (const sample_t *s) ;

C++: void show (const sample_t &s) ;

No copy is made

The compiler rejects any attempt to modify it

Rust: Ownership Moves

```
fn show ( s : Sample ) { ... }
```

```
show ( my_sample ) ;
```

```
show ( my_sample ) ;    error: value moved
```

Passing by value **moves** the instance
into the function

Copy types like i32 and f32 are copied instead

Rust: Borrowing

```
fn show ( s : &Sample ) { ... }
```

```
show ( &my_sample ) ;  
show ( &my_sample ) ; fine
```

& lends the instance **read-only**

Like `const &` in C++, but the `&` is **visible**
at the call

Rust: Mutable Borrowing

```
fn bump ( x : &mut i32 ) {  
    *x += 1 ;  
}
```

```
let mut count = 5 ;  
bump ( &mut count ) ;
```

count is now **6**

&mut at the call says: this may change

The Borrowing Rules

At any moment, a value may have **either**:

any number of `&` borrows (readers)

or

exactly one `&mut` borrow (one writer)

Checked by the compiler, not at run time

Passing Compared

	C	C++	Rust
Copy	<code>int x</code>	<code>int x</code>	<code>x : i32</code>
Read only	<code>const T *x</code>	<code>const T &x</code>	<code>x : &T</code>
Modify	<code>T *x</code>	<code>T &x</code>	<code>x : &mut T</code>
At the call <code>v</code>	<code>&v</code>	<code>v</code>	<code>&v</code> or <code>&mut</code>

Passing a Struct

By value copies **every byte**:

`void show ( sample_t s ) ;` 16 bytes copied

By address copies **one pointer**:

`void show ( const sample_t *s ) ;` 8 bytes

Inside, reach members with `->`

`s->psi`

Returning a Struct

C: `sample_t make_sample ( uint32_t counts ) ;`
C++: `sample make_sample ( uint32_t counts ) ;`
Rust: `fn make_sample ( counts : u32 ) -> Sample`

All three can return a whole struct

The compiler usually builds it directly
in the caller's memory - no extra copy

Passing an Array in C

```
float average ( const float *psi, size_t n ) ;
```

The array **decays** to a pointer to element 0
The length is **lost** - pass it separately

Inside the function, `sizeof(psi)` is **8**
the size of a pointer, not the array

Passing an Array in C++

Fixed size:

```
float average ( const std::array<float, 100>  
&psi ) ;
```

Any size (C++20):

```
#include <span>  
float average ( std::span<const float> psi ) ;
```

Both know their own .size()

Passing an Array in Rust

```
fn average ( psi : &[f32] ) -> f32 {  
    let mut sum = 0.0 ;  
    for i in 0..psi.len() { sum += psi[i] ; }  
    sum / psi.len() as f32  
}
```

```
average ( &readings ) ;
```

&[f32] is a **slice**: pointer and length together

Returning Several Values

C: pass pointers for the outputs
 void min_max (const float *v, size_t n,
 float *lo, float *hi) ;

C, C++: return a struct

Rust: return a tuple
 fn min_max (v : &[f32]) -> (f32, f32)
 let (lo, hi) = min_max (&readings) ;

Same Name, Different Parameters

C++ allows **overloading**:

```
float to_psi ( uint32_t counts ) ;  
float to_psi ( float kpa ) ;
```

The compiler picks by the argument types

C: not allowed - every name is unique

Rust: not allowed - use different names

Default Arguments

C++ only:

```
void show ( float psi, int digits = 1 ) ;
```

```
show ( 50.0f ) ;      digits is 1
```

```
show ( 50.0f, 3 ) ;   digits is 3
```

C and Rust have no default arguments

Data and Functions Belong Together

Many functions exist only to work on **one type**:

sample_init	sample_update
sample_to_kpa	sample_show

The data lives in the struct

The functions live somewhere else

Can we keep them together?

The C Convention

Pass the instance as the **first** parameter:

```
void sample_update ( sample_t *s, uint32_t counts ) {  
    s->adc_counts = counts ;  
    s->psi = adc_to_psi ( counts ) ;  
}
```

```
sample_update ( &my_sample, 1456 ) ;
```

Member Functions

Put the function **inside** the type

It is called **on** an instance:

```
my_sample.update ( 1456 ) ;
```

C++: **member functions**

Rust: **methods**

A Member Function in C++

```
struct sample {  
    uint32_t    adc_counts ;  
    float       psi ;  
    uint64_t    timestamp_ms ;  
  
    float to_kpa ( ) const {  
        return psi * 6.894757f ;  
    }  
} ;
```

Calling a Member Function

```
sample my_sample ;  
sample sample2 ;
```

```
my_sample.to_kpa ( )      uses  
my_sample.psi  
sample2.to_kpa ( )      uses sample2.psi
```

One function, many instances

The Hidden Parameter: **this**

The compiler passes the instance **address** for you

```
my_sample.to_kpa ( )
```

Inside, **this** $\leq$ $\&$ (my_sample)

psi means `this->psi`

Exactly the C convention - just hidden

const Member Functions

```
float to_kpa ( ) const ;
```

const after the parameter list promises
not to change the instance

It may be called on a const instance

Mark every member function that only reads

A Member Function That Modifies

```
struct sample {  
    ...  
    void update ( uint32_t counts ) {  
        adc_counts = counts ;  
        psi = adc_to_psi ( counts ) ;  
    }  
} ;  
  
my_sample.update ( 1456 ) ;
```

Defining It Outside the Struct

sample.h

```
struct sample { ... float to_kpa ( ) const ; } ;
```

sample.cpp

```
float sample::to_kpa ( ) const {  
    return psi * 6.894757f ;  
}
```

:: says which type the function belongs to

Methods in Rust

```
struct Sample { adc_counts : u32, psi : f32, ... }
```

```
impl Sample {  
    fn to_kpa ( &self ) -> f32 {  
        self.psi * 6.894757  
    }  
}
```

Methods go in a separate **impl** block

self is Not Hidden in Rust

`&self` read the instancelike `const` in C++

`&mut self` modify the instance

`self` take ownership of it

You must write **`self.psi`** - never just `psi`

A Rust Method That Modifies

```
impl Sample {  
    fn update ( &mut self, counts : u32 ) {  
        self.adc_counts = counts ;  
        self.psi = adc_to_psi ( counts ) ;  
    }  
}
```

```
let mut my_sample = ... ;  
my_sample.update ( 1456 ) ;
```

The Dot Borrows for You

```
my_sample.update ( 1456 ) ;
```

is the same as

```
Sample::update ( &mut my_sample, 1456 ) ;
```

Which is the C convention again

Constructors

A function that creates an instance

```
C++:    sample ( uint32_t counts )  
        : adc_counts ( counts ),  
        psi ( adc_to_psi ( counts ) ) { }  
sample s ( 1456 );
```

```
Rust:   fn new ( counts : u32 ) -> Sample { ... }  
let s = Sample::new ( 1456 );
```

Functions With No Instance

Belong to the type, but no instance is needed

C++: static member functions
 sample::from_kpa (344.7f)

Rust: associated functions - no self
 Sample::new (1456)

Called with `::` on the **type**, not `.` on an instance

Side by Side

C: `sample_update ( &my_sample, 1456 ) ;`

C++: `my_sample.update ( 1456 ) ;`

Rust: `my_sample.update ( 1456 ) ;`

In all three the function receives the
address of the instance

Public and Private

C++:

struct members **public** by default
class members **private** by default
public: private: labels change it

Rust:

fields and methods are **private** to the module
pub makes them public

Why Hide the Data?

psi must always match adc_counts

If anyone can write `my_sample.psi = 99 ;`
the two disagree

Make the fields **private**

Allow changes only through **update ()**

The type protects its own rules

Functions in the Debugger

step	go into the call
next	run the call, stop after it
finish	run until this function returns
backtrace	show the stack of frames
frame 1	look at the caller's variables

Now the stack in backtrace makes sense

Good Function Habits

Do **one** job

Name it with a verb: update, show,
adc_to_psi

Keep it short enough to see all at once

Few parameters - group the rest in a struct

Pass read-only data as `const` or `&`

What We Can Now Do

- Functions** name a group of statements
 - Parameters** receive arguments by value,
by address, or by borrow
 - Member functions** functions that belong to a
type
- Every call gets its own frame on the stack

Let's practice

Exercise: Using **funcpractice**